SEMANA 6

Miguel Andres Gomez

(Estudiante)

Jesus Ariel Gonzalez Bonilla

(profesor)

20/03/2026

Corhuila

Contenido

Introducción 3

Crear 3 roles de grupo: rol_lectura, rol_escritura, rol_admin. 4

Crear 3 usuarios: u_reportes, u_app, u_admin. 4

Asignar roles a usuarios siguiendo el perfil esperado. 5

Aplicar permisos a DB, esquema, tablas y secuencias (GRANT). 5

Probar (y evidenciar) que: **u_reportes** solo puede SELECT, **u_app** puede INSERT/UPDATE/DELETE, **u_app** NO puede CREATE TABLE. 6

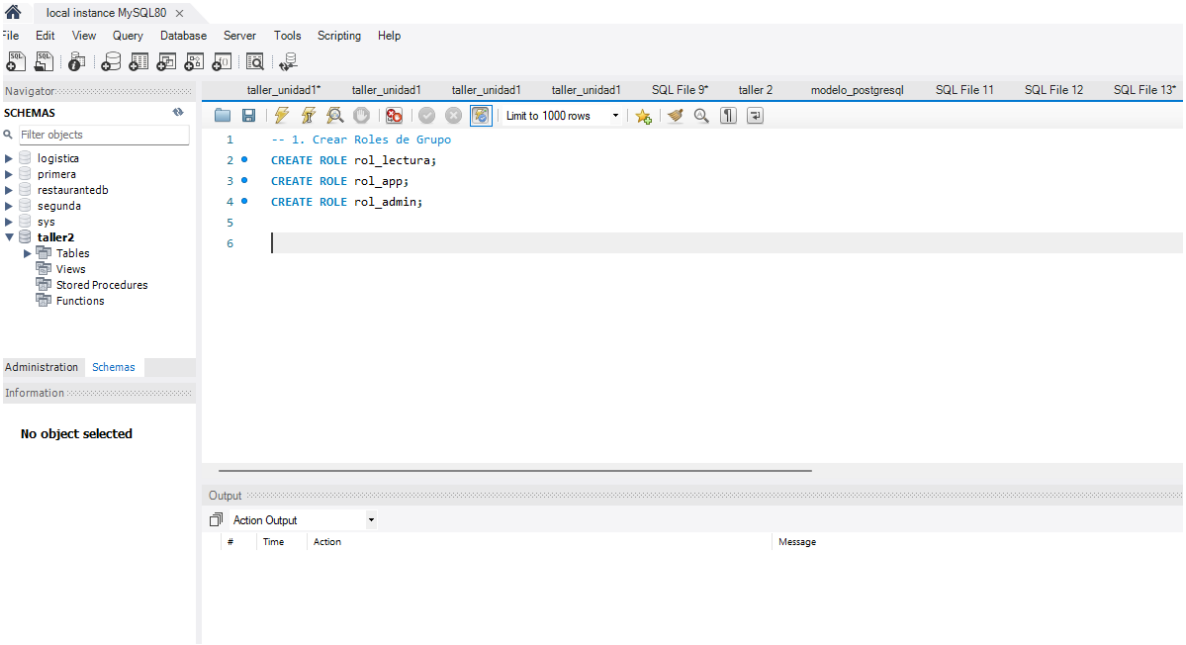
Conclusión 9

Introducción

Se crearon tablas, relaciones, triggers y mecanismos de auditoría, además de aplicar buenas prácticas de seguridad mediante el uso de usuarios, roles y permisos. También se abordaron aspectos de optimización mediante índices para mejorar el rendimiento de las consultas.

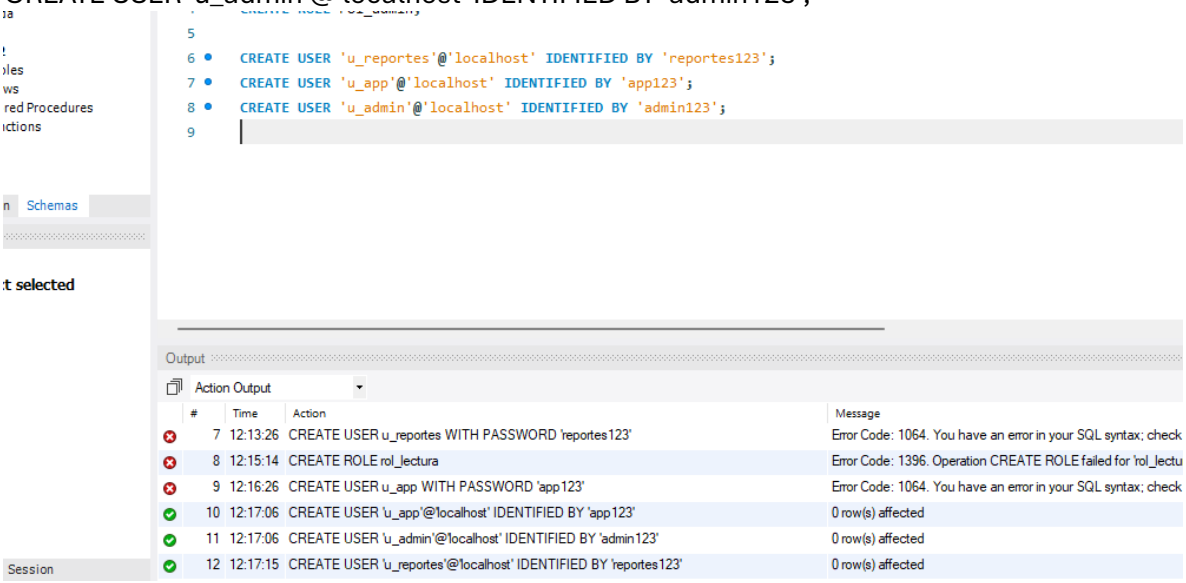
Crear 3 roles de grupo: rol_lectura, rol_escritura, rol_admin.

CREATE ROLE rol_lectura;
CREATE ROLE rol_app;
CREATE ROLE rol_admin;



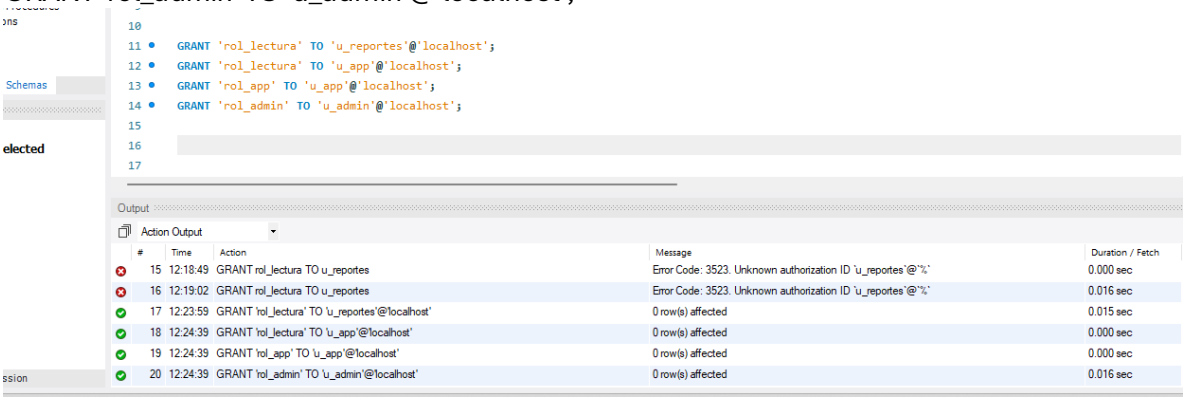
Crear 3 usuarios: u_reportes, u_app, u_admin.

CREATE USER 'u_reportes'@'localhost' IDENTIFIED BY 'reportes123';
CREATE USER 'u_app'@'localhost' IDENTIFIED BY 'app123';
CREATE USER 'u_admin'@'localhost' IDENTIFIED BY 'admin123';



Asignar roles a usuarios siguiendo el perfil esperado.

GRANT 'rol_lectura' TO 'u_reportes'@'localhost';
GRANT 'rol_lectura' TO 'u_app'@'localhost';
GRANT 'rol_app' TO 'u_app'@'localhost';
GRANT 'rol_admin' TO 'u_admin'@'localhost';

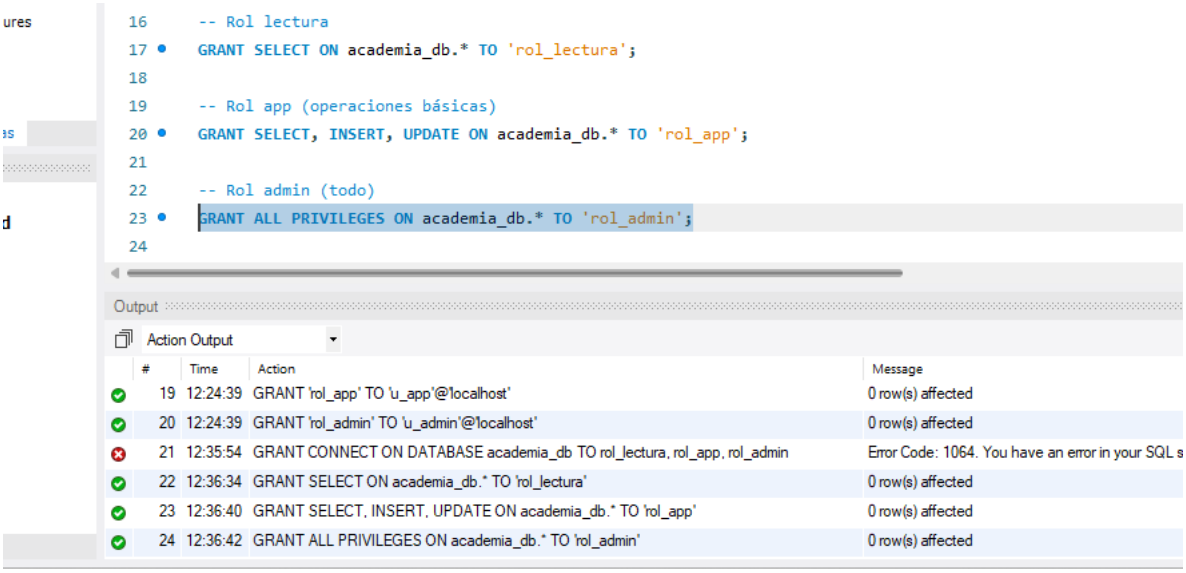


Aplicar permisos a DB, esquema, tablas y secuencias (GRANT).

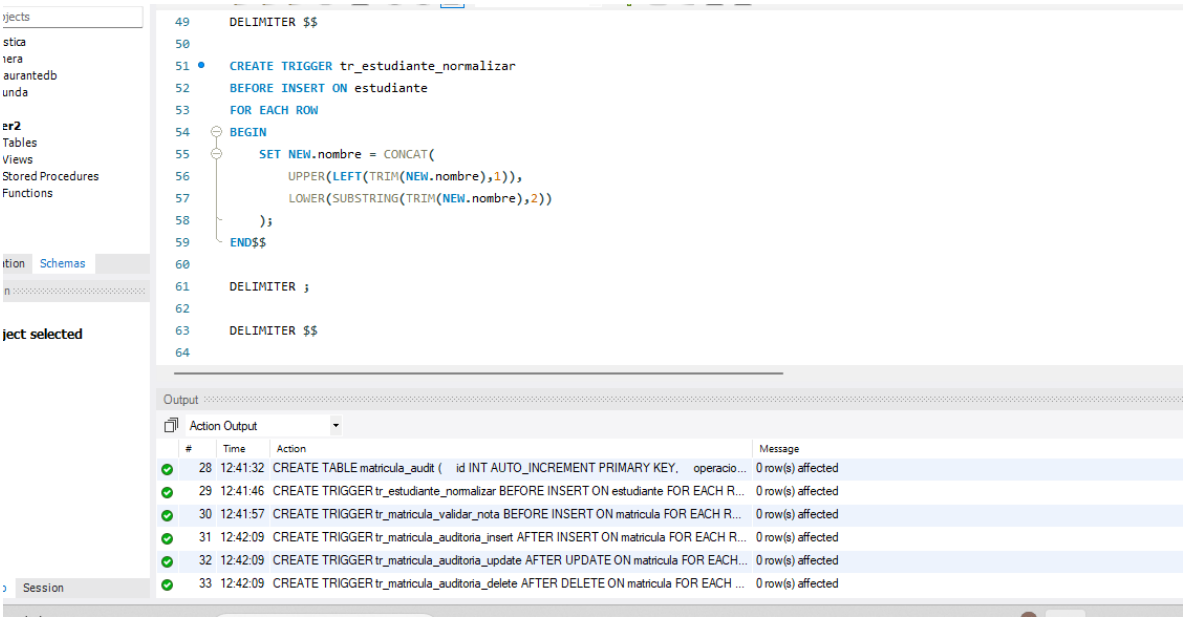
-- Rol lectura
GRANT SELECT ON academia_db.* TO 'rol_lectura';

-- Rol app (operaciones básicas)
GRANT SELECT, INSERT, UPDATE ON academia_db.* TO 'rol_app';

-- Rol admin (todo)
GRANT ALL PRIVILEGES ON academia_db.* TO 'rol_admin';



Probar (y evidenciar) que: **u_reportes** solo puede SELECT, **u_app** puede INSERT/UPDATE/DELETE, **u_app** NO puede CREATE TABLE.




```
CREATE TABLE estudiante (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(255) NOT NULL,
  email VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE matricula (
  id INT AUTO_INCREMENT PRIMARY KEY,
  estudiante_id INT,
  semestre VARCHAR(20) NOT NULL,
  nota_final DECIMAL(3,2),
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (estudiante_id) REFERENCES estudiante(id)
);

CREATE TABLE matricula_audit (
  id INT AUTO_INCREMENT PRIMARY KEY,
  operacion VARCHAR(10),
  matricula_id INT,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  usuario VARCHAR(100)
);
```

```
DELIMITER $$

CREATE TRIGGER tr_estudiante_normalizar
BEFORE INSERT ON estudiante
FOR EACH ROW
BEGIN
  SET NEW.nombre = CONCAT(
    UPPER(LEFT(TRIM(NEW.nombre),1)),
    LOWER(SUBSTRING(TRIM(NEW.nombre),2))
  );
END$$

DELIMITER ;

DELIMITER $$

CREATE TRIGGER tr_matricula_validar_nota
BEFORE INSERT ON matricula
FOR EACH ROW
BEGIN
  IF NEW.nota_final < 0 OR NEW.nota_final > 5 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: nota fuera de rango (0-5)';
  END IF;
END$$

DELIMITER ;

DELIMITER $$

CREATE TRIGGER tr_matricula_auditoria_insert
AFTER INSERT ON matricula
FOR EACH ROW
BEGIN
  INSERT INTO matricula_audit(operacion, matricula_id, usuario)
  VALUES ('INSERT', NEW.id, CURRENT_USER());
END$$

CREATE TRIGGER tr_matricula_auditoria_update
AFTER UPDATE ON matricula
FOR EACH ROW
BEGIN
  INSERT INTO matricula_audit(operacion, matricula_id, usuario)
  VALUES ('UPDATE', NEW.id, CURRENT_USER());
END$$

CREATE TRIGGER tr_matricula_auditoria_delete
AFTER DELETE ON matricula
```

```
FOR EACH ROW
BEGIN
  INSERT INTO matricula_audit(operacion, matricula_id, usuario)
  VALUES ('DELETE', OLD.id, CURRENT_USER());
END$$

DELIMITER ;
```

Conclusión

Se evidenció que el uso de triggers, roles y controles de acceso es fundamental para garantizar la integridad, seguridad y trazabilidad de la información. Además, se reforzó la necesidad de aplicar buenas prácticas para evitar errores y mejorar el desempeño del sistema.